

Business Requirements Document (BRD) & API Specification

Project: Motor Insurance Claims Management Portal

Document Version: 1.0 **Purpose:** Test Automation / RAG Domain Analysis Input

1. Business Overview

The Motor Insurance Claims Management Portal is an enterprise application that enables policyholders to submit motor insurance claims and track their status. Claims adjusters review submitted claims and approve or reject them. Approved claims create payout records that can be monitored by finance managers.

2. User Roles & Actors

Actor	Responsibilities
Policyholder	Submit motor claims, provide incident details, upload supporting documents, and view claim status.
Claims Adjuster	Review submitted claims, move claims to review, approve eligible claims, or reject invalid claims.
Finance Manager	View approved payouts and monitor payout processing status.

3. Business Workflows

BRD-WF-1: Submit Claim

The policyholder submits a new claim with policy_number, claimant_name, claim_amount, incident_date, incident_type, vehicle_registration_number, and description. The system validates mandatory fields and creates the claim with status SUBMITTED.

BRD-WF-2: Review Claim

A claims adjuster opens a SUBMITTED claim and places it IN_REVIEW. Only SUBMITTED claims may enter IN_REVIEW.

BRD-WF-3: Approve Claim

A claims adjuster approves a claim using POST /api/v1/claims/{id}/approve. The claim must currently be SUBMITTED or IN_REVIEW. The system changes the status to APPROVED and automatically creates one payout record.

BRD-WF-4: Reject Claim

A claims adjuster rejects a claim using POST /api/v1/claims/{id}/reject. The claim status changes to REJECTED. No payout record must be created for a rejected claim.

4. Business Validation Rules

- VAL-01: policy_number is mandatory and must not be blank.
- VAL-02: claimant_name is mandatory and must not be blank.
- VAL-03: claim_amount must be greater than 0 and less than or equal to INR 500,000.

- VAL-04: incident_date must not be a future date.
- VAL-05: vehicle_registration_number is mandatory for motor claims.
- VAL-06: A newly submitted claim must automatically receive status SUBMITTED.
- VAL-07: Only SUBMITTED or IN_REVIEW claims may transition to APPROVED.
- VAL-08: Only SUBMITTED or IN_REVIEW claims may transition to REJECTED.
- VAL-09: Approval must create exactly one payout record with payout_amount equal to claim_amount and status PROCESSING.
- VAL-10: Rejection must not create a payout record.

5. API Requirements

Method	Endpoint	Purpose
POST	/api/v1/claims	Submit a new motor insurance claim.
GET	/api/v1/claims	List all claims.
GET	/api/v1/claims/{id}	Retrieve a claim by ID.
PATCH	/api/v1/claims/{id}/review	Move a submitted claim to IN_REVIEW.
POST	/api/v1/claims/{id}/approve	Approve a claim and initiate payout creation.
POST	/api/v1/claims/{id}/reject	Reject a claim without creating a payout.
GET	/api/v1/payouts	List payout records.
GET	/api/v1/payouts/{id}	Retrieve a payout by ID.

6. Expected Request Payload

POST /api/v1/claims

```
{ "policy_number": "POL123456", "claimant_name": "Rahul Sharma", "claim_amount": 75000,
  "incident_date": "2026-07-15", "incident_type": "ACCIDENT", "vehicle_registration_number": "KA01AB1234",
  "description": "Front bumper and headlamp damage" }
```

7. Database Requirements & Side Effects

Table	Important Fields
claims	id, policy_number, claimant_name, claim_amount, incident_date, incident_type, vehicle_registration_number
payouts	id, claim_id, payout_amount, status, created_at

Database Side Effects

- Submitting a valid claim creates one record in the claims table with status SUBMITTED.
- Approving a claim updates its status to APPROVED and creates exactly one linked payout record.
- Rejecting a claim updates its status to REJECTED and creates no payout record.

8. Expected Error Behaviour

- Invalid or missing input returns HTTP 400 with a meaningful validation message.
- A claim ID that does not exist returns HTTP 404.
- Invalid state transitions, such as approving an already REJECTED claim, return HTTP 409.
- Successful claim creation returns HTTP 201; successful reads and state-change operations return HTTP 200.

9. Test Automation Expectations

The intelligent test automation platform should derive positive, negative, boundary, API, workflow, and database assertion scenarios from this BRD. Generated tests must verify both API responses and the corresponding PostgreSQL database state where applicable.